

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



Ética y Seguridad de los Datos

Gestor de Contraseñas Seguro con Criptografía Aplicada

Integrantes del grupo:

Santiago Adolfo Uribe Guerra (100 %)

Docente:

José Carlos Pazos

Mayo 2026,

Barranco

Índice

Índice	1
1. Introducción y Motivación	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Requerimientos Funcionales	2
4. Requerimientos de Seguridad	3
4.1. Protección de datos en reposo	3
4.2. Protección de datos en transporte	3
4.3. Gestión de accesos y autorización	4
4.4. Registros de auditoría (logs)	4
4.5. Estrategias de uso seguro de los datos	4
4.6. Herramientas para análisis de implementación	5
4.7. Identificación de riesgos asociados a la data	5
4.8. Plan de respuesta ante incidentes de seguridad	5
4.9. Recomendaciones de protección de datos futura	6
5. Fundamentación Teórica	6
5.1. Tríada CIA y Ley de Protección de Datos	6
5.2. Principios de Kerckhoffs y Saltzer & Schroeder	6
5.3. Criptografía Simétrica Autenticada: AES-GCM	6
5.4. Derivación de Clave con Argon2	7
5.5. Funciones Hash y HMAC	7
5.6. Control de Acceso y Gestión de Sesiones	7
5.7. Modelado de Amenazas con STRIDE	7
6. Diseño de la Solución	7
6.1. Arquitectura General	7
6.2. Protocolo Criptográfico (Flujo Principal)	8
6.3. Diagrama de Componentes y Flujo de Datos	8
6.4. Interfaz de Usuario (Wireframes)	9
7. Plan de Implementación	9
8. Lecciones Aprendidas y Retrospectiva	9

1. Introducción y Motivación

En la actualidad, las personas gestionan una variedad de cuentas online. La reutilización de contraseñas y su almacenamiento en texto plano (notas digitales o mediante hojas de papel) demuestran una de las vulnerabilidades más comunes, comprometiendo la tríada CIA (Confidencialidad, Integridad y Disponibilidad). Un gestor de contraseñas que aplique criptografía de extremo a extremo (zero-knowledge) resuelve este problema, esto significa que el servidor nunca posee las claves para descifrar los secretos del usuario, alineándose con el principio de Kerckhoffs y la Ley de Protección de Datos Personales del Perú (Ley N° 29733). Este proyecto implementa una aplicación web que, mediante AES-256-GCM, Argon2id y HMAC, protege una bóveda de contraseñas tanto en reposo como en tránsito, cumpliendo los más altos estándares de seguridad computacional.

2. Objetivos

2.1. Objetivo General

Desarrollar una aplicación web segura de gestión de contraseñas que garantice confidencialidad e integridad mediante criptografía aplicada, siguiendo el principio de mínimo privilegio.

2.2. Objetivos Específicos

- Derivar una clave maestra robusta con Argon2id (RFC 9106) a partir de la contraseña del usuario y un secreto de dispositivo.
- Cifrar cada entrada de la bóveda con AES-256-GCM (NIST SP 800-38D) utilizando claves independientes derivadas mediante HKDF.
- Proteger las comunicaciones cliente-servidor con TLS usando certificados digitales autofirmados (mínimo RSA 2048 / ECDSA P-256).
- Implementar autenticación de usuarios con almacenamiento seguro de credenciales (bcrypt/Argon2id para el hash de la cuenta).
- Generar un registro de auditoría con eventos relevantes, excluyendo datos sensibles.
- Aplicar modelado de amenazas (STRIDE) y análisis de riesgos para anticipar brechas de seguridad.

3. Requerimientos Funcionales

1. **RF1 – Registro y autenticación de usuario:** el usuario puede crear una cuenta con nombre de usuario y contraseña (esta última nunca se emplea para cifrar la bóveda, solo para acceder al servicio).
2. **RF2 – Configuración de la contraseña maestra:** durante el primer uso, el usuario establece una frase maestra que se deriva localmente y cuya verificación se realiza mediante un hash enviado al servidor.

3. **RF3 – Desbloqueo de la bóveda:** al iniciar sesión, el usuario proporciona la contraseña maestra, el frontend deriva la clave, solicita el blob cifrado al servidor y descifra la bóveda en memoria.
4. **RF4 – CRUD de entradas:** añadir, visualizar, editar y eliminar sitios web, nombres de usuario y contraseñas dentro de la bóveda.
5. **RF5 – Generador de contraseñas seguras:** herramienta para crear contraseñas aleatorias con longitud y composición configurables, utilizando un CSPRNG.
6. **RF6 – Análisis de fortaleza:** indicador visual de la robustez de la contraseña maestra y de las almacenadas (basado en entropía y listas de contraseñas comunes).
7. **RF7 – Sincronización y respaldo:** la bóveda cifrada se almacena en el servidor y puede ser descargada localmente como archivo de respaldo.
8. **RF8 – Bloqueo automático:** tras un periodo de inactividad, la sesión se bloquea borrando las claves de memoria y solicitando de nuevo la contraseña maestra.

4. Requerimientos de Seguridad

4.1. Protección de datos en reposo

- **Cifrado de la bóveda:** AES-256-GCM. La clave se deriva con Argon2id. Cada entrada posee un nonce único de 12 bytes y una clave independiente expandida con HKDF-SHA256.
- **Hash de verificación de la maestra:** Para comprobar la corrección de la contraseña maestra sin exponerla, se almacena en el servidor un hash derivado con Argon2id usando un salt distinto. El frontend envía el hash calculado, luego el servidor lo compara.
- **Credenciales de la cuenta:** El hash de la contraseña de autenticación se almacena con bcrypt o Argon2id, siguiendo las recomendaciones de OWASP.
- **Base de datos:** La base de datos del servidor solo guarda datos cifrados o hasheados. Nunca se almacenan claves de cifrado ni secretos en claro.

4.2. Protección de datos en transporte

- **HTTPS con TLS 1.3:** Se emplea un certificado digital autofirmado (ECDSA P-256 o RSA 2048) generado con OpenSSL. El frontend verifica la cadena de confianza anclando el certificado de la CA propia en el almacén de confianza del navegador.
- **Política de cookies seguras:** Las cookies de sesión se marcan con los atributos HttpOnly, Secure y SameSite=Strict para mitigar XSS y CSRF.
- **Encabezados de seguridad HTTP:** Se incluyen Content-Security-Policy, X-Content-Type-Options, Strict-Transport-Security y X-Frame-Options.

4.3. Gestión de accesos y autorización

- **Modelo RBAC básico:** se definen roles de usuario (por ahora un único rol de propietario). El servidor aplica control de acceso verificando que el identificador de usuario de la petición coincida con el del token JWT.
- **Principio de mínimo privilegio:** la cuenta del servicio de base de datos tiene permisos limitados exclusivamente a las operaciones necesarias.
- **Mecanismo de bloqueo de cuenta:** tras 5 intentos fallidos consecutivos, la cuenta se bloquea temporalmente (15 minutos), registrándose en logs.

4.4. Registros de auditoría (logs)

- **Eventos registrados:** inicios de sesión exitosos y fallidos, operaciones de sincronización de la bóveda, cambios en la cuenta, errores del sistema.
- **Contenido excluido:** nunca se registran contraseñas, contenidos de la bóveda (ni siquiera cifrados) ni datos personales.
- **Protección de logs:** se almacenan en un archivo con permisos restringidos (solo lectura para el proceso servidor) y se rotan para evitar llenado de disco.
- **Monitoreo:** se describe cómo se podría integrar un SIEM (no implementado en esta fase) para detección de anomalías.

4.5. Estrategias de uso seguro de los datos

Políticas:

- La contraseña maestra no debe reutilizarse para la cuenta de la aplicación.
- Se exige una longitud mínima de 8 caracteres y se muestra un medidor de fortaleza.
- Se recomienda activar autenticación de dos factores (posible mejora futura).

Procedimientos:

- El administrador se compromete a no examinar los logs a menos que sea necesario para atender un incidente.
- Se instruye al usuario para que realice copias de respaldo locales periódicas de su bóveda.

Concientización y formación: en la interfaz se mostrarán mensajes educativos sobre la importancia de no compartir la contraseña maestra y de la responsabilidad del usuario sobre su secreto.

4.6. Herramientas para análisis de implementación

- **OWASP ZAP / Burp Suite Community Edition:** para escaneo pasivo y activo de la API, buscando vulnerabilidades como las del OWASP Top 10 (A03: Inyección, A05: Configuración incorrecta de seguridad).
- **npm audit / Snyk:** para analizar la cadena de suministro de dependencias (A03: Software Supply Chain Failures).
- **Test vectors de NIST:** se compararán las salidas de AES-GCM y Argon2id con vectores de prueba publicados por NIST, garantizando la correctitud de la implementación criptográfica.
- **ESLint con plugins de seguridad:** para identificar patrones de código inseguros en el frontend (manejo de innerHTML).

4.7. Identificación de riesgos asociados a la data

Se realizó el marco STRIDE para modelar las amenazas principales:

Tabla 1: Análisis de amenazas con STRIDE

Categoría STRIDE	Amenaza	Entidades afligidas
Spoofing	Suplantación del servidor mediante MITM	Usuarios (posible robo de credenciales)
Tampering	Modificación del blob cifrado en el servidor	Usuarios (pérdida de integridad de la bóveda)
Repudiation	Usuario niega haber realizado una acción	Operador del servicio (falta de evidencia)
Information Disclosure	Fuga del archivo de la bóveda por SQLi o acceso indebido	Usuarios (aunque cifrado, expone a ataques offline si la maestra es débil)
Denial of Service	Ataque de denegación de servicio que impide la sincronización	Usuarios (pérdida de disponibilidad)
Elevation of Privilege	Compromiso de la cuenta de administración del servidor	Todos los usuarios (posibilidad de modificar el frontend o robar hashes)

4.8. Plan de respuesta ante incidentes de seguridad

Ante una sospecha de fuga o pérdida de datos, se seguirá el siguiente protocolo básico (inspirado en NIST SP 800-61):

1. **Detección:** los logs de acceso son monitoreados diariamente (script automatizado que alerta por múltiples intentos fallidos o descargas masivas).
2. **Contención:** deshabilitar temporalmente el servidor, revocar todos los tokens JWT activos y cambiar las claves de API internas.

3. **Erradicación:** identificar la causa raíz y aplicar un parche correctivo. Revisar la integridad del código frontend desplegado.
4. **Recuperación:** restaurar el servicio con la versión corregida. Notificar a los usuarios afectados por correo electrónico recomendando el cambio de la contraseña maestra.
5. **Lecciones aprendidas:** documentar el incidente y ajustar los controles preventivos.

4.9. Recomendaciones de protección de datos futura

- **Autenticación multifactor (MFA):** implementar TOTP o FIDO2/WebAuthn para la cuenta de la aplicación, desacoplando el acceso al servicio del conocimiento de la contraseña maestra.
- **Hardware Security Module (HSM):** almacenar las claves privadas del certificado TLS en un módulo dedicado, evitando su exposición en el sistema de archivos.
- **Auditoría externa y bug bounty:** someter la aplicación a un pentest profesional y abrir un programa de recompensas para la comunidad de seguridad.
- **Respaldo cifrado en la nube:** ofrecer integración con servicios de almacenamiento en la nube (Google Drive, Dropbox) para que el usuario sincronice su bóveda cifrada de manera redundante, aplicando cifrado del lado del cliente.

5. Fundamentación Teórica

5.1. Tríada CIA y Ley de Protección de Datos

La seguridad de la información se define mediante la confidencialidad (los secretos solo son accesibles por el dueño), integridad (no han sido alterados) y disponibilidad (se accede cuando se necesita). En el Perú, la Ley N° 29733 otorga al titular de los datos personales los derechos ARCO (Acceso, Rectificación, Cancelación y Oposición), por lo que el sistema debe garantizar que el usuario controle exclusivamente sus contraseñas.

5.2. Principios de Kerckhoffs y Saltzer & Schroeder

El principio de Kerckhoffs establece que la seguridad debe residir en la clave y no en el secreto del algoritmo. Por ello, todos los algoritmos empleados (AES, Argon2) son públicos y estandarizados. Además, el diseño se adhiere a los principios de Saltzer y Schroeder: mínimo privilegio (cada proceso accede solo a lo necesario), seguridad por defecto (las entradas se validan estrictamente), y diseño abierto (el código será revisable).

5.3. Criptografía Simétrica Autenticada: AES-GCM

AES (Advanced Encryption Standard, FIPS 197) es un cifrado por bloque ampliamente analizado. Usado en modo GCM (Galois/Counter Mode, NIST SP 800-38D) provee confidencialidad, integridad y autenticación en una sola operación (AEAD). GCM utiliza un contador para generar un flujo de claves y una función GHASH para el tag de autenticación, garantizando resistencia contra ataques de texto escogido. Se emplearán claves de 256 bits y nonces de 96 bits generados aleatoriamente.

5.4. Derivación de Clave con Argon2

Argon2, ganador del Password Hashing Competition, ofrece resistencia contra ataques de fuerza bruta con hardware especializado (GPU/ASIC) gracias a su uso intensivo de memoria. Se utilizará la variante Argon2id, que combina los modos dependiente e independiente de datos, con parámetros que demandan 64 MiB de memoria y 3 iteraciones, lo que ralentiza significativamente cualquier intento masivo de adivinación.

5.5. Funciones Hash y HMAC

Para verificar la integridad y autenticidad de mensajes se utiliza HMAC-SHA256 (RFC 2104), que combina una clave secreta con una función hash resistente a colisiones (SHA-256). Se emplea en la expansión de claves (HKDF) y podría usarse para firmar metadatos de la bóveda.

5.6. Control de Acceso y Gestión de Sesiones

Las contraseñas de la cuenta se almacenan con algoritmos lentos (bcrypt o Argon2id) y salt único. Las sesiones se gestionan mediante cookies seguras con atributos HttpOnly, Secure y SameSite. El sistema emplea JWT firmados con HMAC-SHA256 para las peticiones a la API, con tiempo de expiración corto (1 hora) y renovación mediante refresh tokens (opcional).

5.7. Modelado de Amenazas con STRIDE

STRIDE es un marco de Microsoft para categorizar amenazas en: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service y Elevation of Privilege. Su aplicación en la fase de diseño permite anticipar vulnerabilidades antes de escribir código, en línea con la metodología de threat modeling.

6. Diseño de la Solución

6.1. Arquitectura General

La aplicación sigue una arquitectura cliente-servidor clásica:

- **Frontend:** Vanilla JS + `libsodium-wrappers` y `argon2-browser`. Realiza toda la criptografía en el navegador, se comunica con el *backend* exclusivamente por HTTPS.
- **Backend:** API REST con Node.js/Express. Componente sin estado (*stateless*), aislado de las claves de cifrado, solo valida el *hash* de verificación y gestiona los *blobs* ya cifrados.
- **Base de datos:** PostgreSQL. Almacena registros de usuario, *hashes* (bcrypt), *salts* y el *blob* cifrado de la bóveda. Se accede mediante `pg` (*pool* de conexiones).
- **Certificados digitales:** CA propia generada con OpenSSL, el certificado de la CA se instala manualmente en el cliente.

6.2. Protocolo Criptográfico (Flujo Principal)

1. **Registro de cuenta:** el usuario elige **username** y **password**. El frontend envía **username** y el hash de la contraseña (bcrypt) al servidor, que almacena el hash y un salt único.
2. **Configuración de la bóveda:** el usuario ingresa la **contraseña maestra PM**. El frontend:
 - Genera un salt aleatorio $S_{maestra}$.
 - Calcula $K_{verify} = \text{Argon2id}(PM, S_{maestra})$ y lo envía al servidor junto con $S_{maestra}$.
 - Deriva la clave de cifrado maestra $K_{master} = \text{Argon2id}(PM, S_{maestra} || "enc")$ (mediante dos llamadas o HKDF).
3. **Primer uso / descarga:** el servidor devuelve un blob vacío. El frontend crea la estructura de la bóveda vacía, la cifra con K_{master} y un nonce fresco, y la envía al servidor.
4. **Desbloqueo posterior:**
 - a) El usuario ingresa PM' . El frontend obtiene $S_{maestra}$ del servidor (previa autenticación básica) y calcula $K'_{verify} = \text{Argon2id}(PM', S_{maestra})$.
 - b) Envía K'_{verify} al endpoint de verificación. Si coincide, el servidor retorna el blob cifrado de la bóveda.
 - c) El frontend deriva K_{master} y descifra el blob con AES-GCM, verificando el tag.
5. **Operaciones CRUD:** las entradas se manipulan en memoria. Al guardar, se cifra nuevamente toda la bóveda con un nonce fresco y se sube el nuevo blob al servidor.
6. **Bloqueo:** al cerrar sesión o tras inactividad, se borran PM , K_{master} y los datos descifrados de la memoria (sobrescribiendo variables).

6.3. Diagrama de Componentes y Flujo de Datos

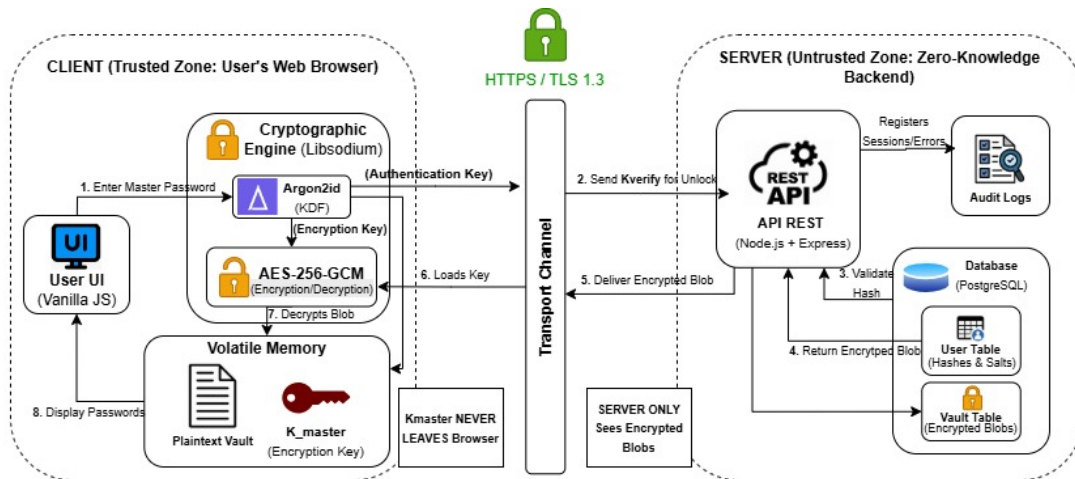


Figura 1: Diagrama de Arquitectura del gestor de contraseñas.

6.4. Interfaz de Usuario (Wireframes)

- **Login / Registro:** formulario con nombre de usuario y contraseña.
- **Desbloqueo:** campo para la contraseña maestra, indicador de fortaleza y botón de desbloqueo.
- **Lista de entradas:** tabla con nombre del sitio, usuario y botón para copiar contraseña. Campo de búsqueda por nombre.
- **Formulario de entrada:** campos sitio, nombre de usuario, contraseña (con generador integrado) y notas.
- **Configuración:** opción para exportar la bóveda, cerrar sesión, cambiar contraseña de cuenta.

La interfaz se diseñará siguiendo el principio de aceptabilidad psicológica, siendo intuitiva para que el usuario no busque atajos inseguros.

7. Plan de Implementación

La primera entrega (este documento) comprende el diseño y la especificación completa. La implementación se desarrollará en la segunda fase, siguiendo este cronograma:

1. **Semana 1:** configuración del entorno (repositorio Git, OpenSSL, Node.js + Express, estructura base del frontend HTML/CSS/JS).
2. **Semana 2:** implementación del registro, login y gestión de sesiones con cookies seguras.
3. **Semana 3:** integración de las bibliotecas criptográficas en el frontend, flujo de configuración de la bóveda y verificación de la maestra.
4. **Semana 4:** cifrado/descifrado de la bóveda, operaciones CRUD y sincronización con el servidor.
5. **Semana 5:** generador de contraseñas, medidor de fortaleza, bloqueo automático y logs.
6. **Semana 6:** pruebas de seguridad con OWASP ZAP, revisión de dependencias y corrección de hallazgos.
7. **Semana 7:** documentación final, preparación de la presentación y retrospectiva.

8. Lecciones Aprendidas y Retrospectiva

Se reflexiona sobre la planificación:

- La elección de un caso de uso acotado pero realista (gestor de contraseñas) permite aplicar múltiples conceptos criptográficos sin que el desarrollo se nos vaya de las manos.

- La revisión de las librerías disponibles (libsodium, argon2-browser) fue crucial, se verificó que cuentan con auditorías de seguridad y una API que minimiza el riesgo de mal uso.
- Se identificó la importancia de no implementar primitivas criptográficas propias, adhiriéndose al principio de “no ruedes tu propia criptografía”.
- El modelado de amenazas con STRIDE reveló riesgos que inicialmente no se habían considerado (por ejemplo, la modificación del blob en el servidor), lo que llevó a reforzar la verificación de integridad con GCM y a considerar firmas digitales adicionales.